

# Day 1: Structured Outputs and Pydantic

**Difficulty:** beginner

## Concept

Structured outputs mean getting information from an AI in a predictable, organized format, like JSON, instead of just free-form text. This makes it much easier for other computer programs to understand and use the AI's response. Pydantic is a powerful Python library that helps us define exactly what that organized format should look like (a schema) and automatically checks if the AI's output matches our expectations, ensuring data quality and consistency.

## Theory

**Why Structured Outputs are Useful:** Imagine asking an AI for a list of ingredients for a recipe. If it gives you a paragraph of text, your program would have to try and guess which words are ingredients and which are quantities. If it gives you a structured output, like a JSON list of objects where each object has 'item' and 'quantity' keys, your program can easily read and process that information without errors. This reliability is crucial for building robust AI applications.

**How Pydantic Helps:** Pydantic allows you to define data models using standard Python type hints. For example, you can say 'this field must be a string' or 'this field must be an integer'. When you receive data (like a JSON string from an AI), Pydantic can parse it and automatically validate it against your defined model. If the data doesn't match the types or structure you specified, Pydantic will raise an error, immediately telling you there's a problem. This saves a lot of time and prevents bugs that might arise from malformed AI responses.

**How it Works with LLMs:** When you prompt an LLM, you can instruct it to return its answer in a specific structured format, often JSON. For example, you might say, 'Extract the person's name and age from the following text and return it as a JSON object with keys 'name' and 'age'.' Once the LLM provides that JSON string, you use Pydantic to load and validate it, turning it into a Python object you can easily work with.

## Practical Task

**Practical Example:** Let's say you want an AI to extract information about a product. You expect a product name (text), a price (a number), and whether it's in stock (true/false). Here's how you'd define a Pydantic model for this:

```
pythonfrom pydantic import BaseModelclass ProductInfo(BaseModel):    name: str    price: float    in_stock: bool# Imagine an LLM outputted this JSON string:llm_output_json_string = '{"name": "Wireless Mouse", "price": 25.99, "in_stock": true}'# To use Pydantic, you first parse the JSON string into a Python dictionary:import jsonproduct_data = json.loads(llm_output_json_string)# Now, create an instance of your Pydantic model:product = ProductInfo(**product_data)# You can now access the data with type safety:print(f"Product Name: {product.name}")print(f"Price: ${product.price}")print(f"In Stock: {product.in_stock}")# If the LLM outputted something like '{"name": "Keyboard", "price": "thirty", "in_stock": "yes"}', Pydantic would raise a validation error because 'thirty' is not a float and 'yes' is not a boolean.
```

**Practical Exercise:** Your task is to define a Pydantic model for 'WeatherReport'. This model should include:

1. `city`: a string for the city name.
2. `temperature`: an integer for the temperature in Celsius.
3. `condition`: a string describing the weather (e.g., 'sunny', 'cloudy', 'rainy').
4. `humidity`: an integer for the humidity percentage.

After defining the model, write a Python snippet that shows how you would attempt to parse the following (hypothetical) JSON string from an LLM into your `WeatherReport` model:

```
json{"city": "London", "temperature": 15, "condition": "cloudy", "humidity": 80}
```

Explain in a comment what would happen if the `temperature` was given as a string like "fifteen" instead of an integer.

## Quiz

**1 What is the primary goal of using structured outputs from an AI?**

Answer: To make AI responses predictable, organized, and easy for other computer programs to process reliably.

**2 What is Pydantic used for in the context of structured outputs?**

Answer: Pydantic is used to define a schema (expected format and types) for data and to validate incoming data (like JSON from an AI) against that schema, ensuring data quality.

**3 If an AI provides a JSON output where a field expected to be an integer is instead a string, what will Pydantic do when you try to parse it?**

Answer: Pydantic will raise a validation error, indicating that the data does not match the expected type defined in the model.